

ASIC 期末课程设计:

基于 Basys3 的人体反应速率检测系统

徐梓严 23300756004

2026 年 6 月 20 日

復旦大學



目录

1 报告和演示视频获取地址	3
2 设计规则	3
2.1 设计要求	3
2.2 设计思路	3
2.2.1 设计平台	3
2.2.2 设计流程图	3
3 设计实现	5
3.1 顶层架构与模块划分	5
3.2 各模块和验证	6
3.2.1 状态机 state.sv (主 FSM)	6
3.2.2 按键消抖 debounce_counter.sv / debouncer.sv	9
3.2.3 线性反馈移位寄存器 lfsr.sv	9
3.2.4 7 段数码管驱动 seven_seg.sv	10
3.2.5 蜂鸣器驱动 buzzer.sv	10
3.2.6 7 段字形查表与 BCD 转码	11
3.2.7 7 段显示时分复用	13
3.2.8 历史记录浏览	13
4 仿真与测试	14
4.1 SystemVerilog testbench	14
4.2 justfile 工作流	15
4.3 测试波形图	16
4.4 FPGA 板介绍	17
4.5 引脚分配 (XDC 摘要)	17
4.6 综合与实现	18
4.6.1 综合条件	18
4.6.2 资源利用率	18
4.6.3 时序结果	19
4.6.4 功耗	19
4.7 上板结果展示	20
4.8 设计总结	24

一 报告和演示视频获取地址

项目仓库 (含源代码、报告 PDF、波形)

<https://github.com/SuZiYan611/2026ASIC>

演示视频

<https://github.com/SuZiYan611/2026ASIC/blob/main/作嘍演示.mp4>

二 设计规则

2.1 设计要求

设计一套用于测试人体反应速率的系统。用户需要根据系统状态变化做出相对应的动作。系统通过计算用户的反应时间得到其反应速率。

2.2 设计思路

2.2.1 设计平台

本次采用了 Digilent Basys3 开发板, 搭载 Xilinx Artix-7 xc7a35tcpg236-1。系统主时钟由板载 100 MHz 晶振提供, 直接接入全局时钟网络 W5, 无需 PLL 即可工作。

人机交互外设:

- 显示: 4 位共阳极 7 段数码管 (an[3:0] 低电平选通 + seg[7:0] 低电平点亮), 由 seven_seg 模块以时分复用方式扫描, 并通过 brightness 输入实现 PWM 亮度调节
- 输入: 5 个方向按键 (btn_center / btn_up / btn_down / btn_left / btn_right), 按键经独立消抖后参与状态转移
- 指示: 16 个独立 LED (led[15:0], 高电平点亮) + 蜂鸣器 (buzz, PMOD JA1)

开发工具链:

- 仿真与验证: Verilator 编译 SystemVerilog
- 综合与实现: Vivado 2025.2 (约束文件 cons.xdc)
- 下载: openFPGALoader (just upload)

2.2.2 设计流程图

本系统主要用于测试用户的反应速率。系统运行过程中, 会随机改变当前状态 (数码管显示由滚动 8 切换为 _GO_), 并要求用户在限定时间内做出对应操作。系统通过记录状态变化到用户输入之间的时间差, 计算用户的反应时间。

系统整体运行流程如图所示, 主要包括以下几个阶段:

1. 系统初始化 (按键消抖、毫秒定时器、LFSR 种子装载)
2. 等待用户开始 (IDLE 状态, 数码管滚动显示 8)
3. 随机延时阶段 (INIT 状态: 4 次淡入/淡出动画 → 空白 → 进入 TRIGGERED)
4. 触发阶段 (TRIGGERED: 滚动 8 倒计时 → 倒计时结束进入 MEASURING)
5. 测量阶段 (MEASURING: 毫秒计数器递增, 等待 btn_center)
6. 结果显示 (MEASURED: BCD 转码后以 4 位十进制毫秒值显示, 可浏览历史)

7. 蜂鸣器按反应时间播放不同曲调 (tune 0-4)

系统上电后, 按键消抖器和毫秒定时器开始工作, 状态机进入 IDLE, 4 位数码管轮询显示 8 (无小数点, 全亮), 起到“待测”提示作用。

当用户按下 `btn_center` 后, 状态机进入 INIT。该阶段分三个子相位: `FADE_IN` (亮度 0→15, 共 $20 \times 15 = 300$ ms 渐入)、`FADE_OUT` (亮度 15→0 渐出)、`BLANK_PHASE` (200 ms 全灭)。这样设计的目的是制造“系统在思考”的视觉提示, 避免用户预判起始时刻。

`BLANK_PHASE` 结束后状态机进入 `TRIGGERED`, 同时装载 LFSR 产生的 2-5 s 随机延时 `countdown_val`, 数码管继续滚动 8。随机延时计数到 0 后, 进入 `SHOW_GO` 子相位, 数码管中央两位显示 GO (其余熄灭) 作为反应启动信号, 并启动 500 ms 短延时后切到 `MEASURING`。

`MEASURING` 状态下, 毫秒计数器 `ms_elapsed` 每 1 ms 加 1, 数码管保持熄灭。用户必须在 5000 ms 之内按下 `btn_center`, 系统记录 `ms_elapsed` 作为本次反应时间; 若超时则标记 `timed_out`。

进入 `MEASURED` 后, 反应时间通过 double-dabble 算法转为 4 位 BCD, 并以十进制毫秒值显示在 4 位数码管上。`btn_up` / `btn_down` 可在最近 8 次历史记录之间切换。蜂鸣器播放对应曲调: tune 0 (小于 200 ms, 最快) → tune 3 (大于等于 400 ms), 超时为 tune 4。按下 `btn_center` 重新进入 IDLE, 开始新一轮测试。

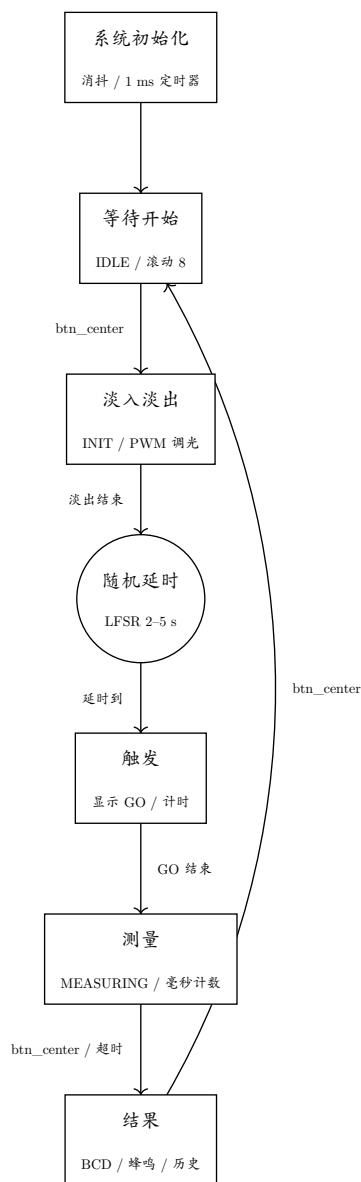


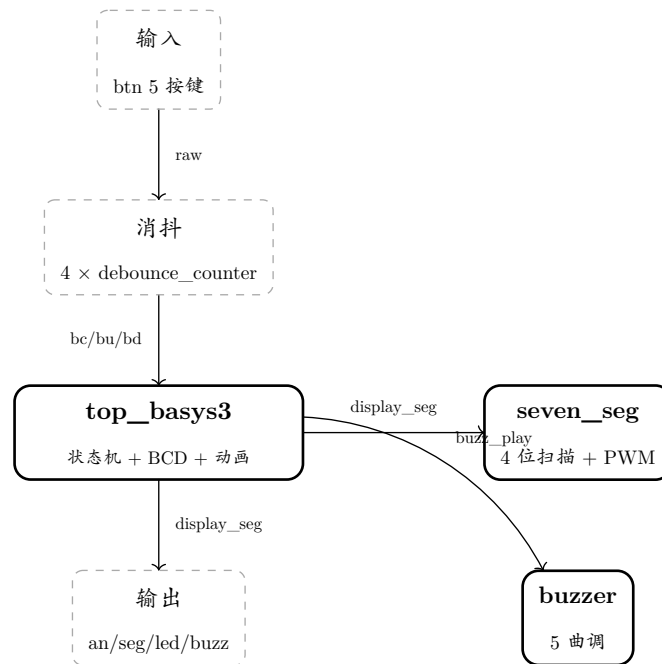
图 1 系统设计流程图（自顶向下）

三 设计实现

3.1 顶层架构与模块划分

整个系统由 `top_basys3.sv` 中的 `top_basys3` 模块统一编排，按功能可划分为 4 个层次：

1. 输入层：5 个方向按键经独立消抖后产生 `bc` / `bu` / `bd` 等边沿脉冲
2. 控制层：5 状态游戏 FSM（`IDLE` / `INIT` / `TRIGGERED` / `MEASURING` / `MEASURED`）配合毫秒定时器、LFSR 随机数与历史缓冲区
3. 驱动层：`seven_seg`（4 位时分复用 + PWM 调光）、`buzzer`（5 种曲调）
4. 输出层：4 位数码管（`an` + `seg`）、16 个 LED（`led`）、蜂鸣器（`buzz`）



`top_basys3` 内联实现了以下子逻辑（外部 *.sv 辅助模块保留以便仿真复用）：

- 计时：MS_CNT（100 MHz 下 100 000 周期 = 1 ms 滴答）
- 随机数：内联 16-bit LFSR（多项式 $x^{16} + x^{14} + x^{13} + x^{11} + 1$ ），种子 16'hFADE，在 IDLE 持续滚动
- 消抖：内联 3 路按键消抖（DB_MAX = 10 ms 状态机）
- 显示：内联 4-digit BCD 转码（double-dabble）+ 7 段字形查表
- 驱动：seven_seg.sv（4 位扫描 + PWM 亮度）
- 音效：buzzer.sv（5 种曲调，频率分频器 + ms 节拍）

3.2 各模块和验证

3.2.1 状态机 state.sv（主 FSM）

系统核心 FSM 抽到独立模块 state.sv（共 6 个状态），由 top_basys3 实例化名为 system_state。ready 是综合后的边沿/事件信号（按键 bc_rise、倒计时完成 countdown_done、测量结束 ms_done_pulse、超时 bi_rise[3] 等），halt 由 top_basys3 在 TRIGGERED 阶段检测到 ≥ 5 次 spam 按键时拉高。

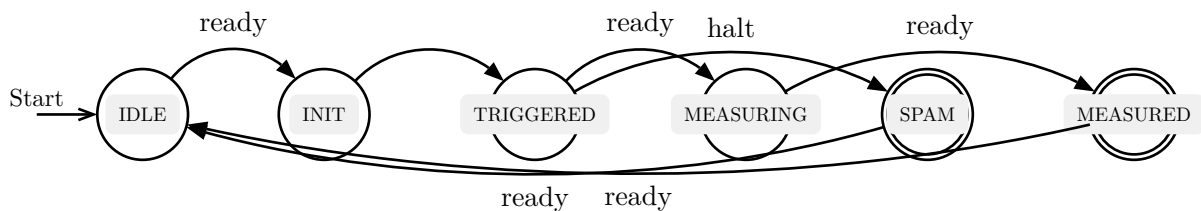


图 3 状态转移图（state.sv 6 状态 FSM）

整体状态转换图如图 3 所示。INIT 状态承担视觉提示与延时锁存两重作用：4 段亮度渐变动画结束后锁存 LFSR 给出的随机延时 random_delay。TRIGGERED 状态下先做随机延时倒计时，倒计时

结束切到 SHOW_GO 子相位显示 _GO_ 字样再进入 MEASURING, 以确保用户有明确的反应起点。MEASURING 计算反应时间, 单位为毫秒, 停止后进入 MEASURED 显示 BCD 结果并支持历史浏览。

作弊拦截依旧保留: 在 TRIGGERED 阶段, top_basys3 通过 spam_count 累计 bc_rise 次数, 达到 5 次后拉高 halt, 由 state.sv 跳转到 SPAM 并显示警告; display_pipeline.sv 会在 SPAM 下渲染 SPAMMED! 提示。修复流程为 bc_rise[3] (即 btn_left) → IDLE。

top_basys3 中与历史记录相关的核心转移代码如下:

```
MEASURING: begin
    if (ready && !ms_done) begin
        measured_latency <= ms_elapsed;
        history[0] <= ms_elapsed;
        for (int i = 1; i < HIST_DEPTH; i++) history[i] <= history[i-1];
        if (history[HIST_DEPTH-1] == 0) begin
            for (int i = 1; i < HIST_DEPTH; i++) history[i] <= ms_elapsed;
            avg_latency <= ms_elapsed;
        end else begin
            avg_latency <= avg_latency + (ms_elapsed / HIST_DEPTH) -
                (history[HIST_DEPTH-1] / HIST_DEPTH);
        end
        hist_wr <= hist_wr + 1;
        hist_rd <= 0;
    end
    if (ms_done_pulse && !bi[0] && !timed_out) begin
        timed_out <= 1;
        history[0] <= ms_elapsed;
        for (int i = 1; i < HIST_DEPTH; i++) history[i] <= history[i-1];
        if (history[HIST_DEPTH-1] == 0) begin
            for (int i = 1; i < HIST_DEPTH; i++) history[i] <= ms_elapsed;
            avg_latency <= ms_elapsed;
        end else begin
            avg_latency <= avg_latency + (ms_elapsed / HIST_DEPTH) -
                (history[HIST_DEPTH-1] / HIST_DEPTH);
        end
        hist_wr <= hist_wr + 1;
        hist_rd <= 0;
    end
end
```

代码 1 MEASURING 状态转移与历史记录写入

```

MEASURED: begin
    if (bi_rise[1] || bi_rise[2]) begin
        hist_idle <= 0;
        if (bi_rise[1]) begin
            if (hist_rd > 0) hist_rd <= hist_rd - 1;
            else if (hist_wr > 0) hist_rd <= hist_wr - 1;
        end
        if (bi_rise[2]) begin
            if (hist_rd + 1 < hist_wr) hist_rd <= hist_rd + 1;
            else hist_rd <= 0;
        end
    end else if (hist_rd != 0 && ms_passed) begin
        if (hist_idle >= 200) begin // 2s real, ~200ms SIM
            hist_rd <= 0;
            hist_idle <= 0;
        end else begin
            hist_idle <= hist_idle + 1;
        end
    end
end
end

```

代码 2 MEASURED 状态下历史记录浏览

模块顶层定义如下:

```

module top_basys3 #(
    parameter SIM_MODE = 0,
    parameter CLK_HZ   = 100000000
)(
    input clk,
    input rst,
    input btn_center,
    input btn_up,
    input btn_down,
    input btn_left,
    output logic [3:0] an,
    output logic [7:0] seg,
    output logic [15:0] led,
    output buzz
);

```


验证: SystemVerilog testbench `tests/tb_top_basys3.sv` (详见后文) 覆盖 6 状态全循环、bc 长按不重复触发、bu/bd 历史翻页、MEASURING 超时回退到 MEASURED、spam_count ≥ 5 触发 SPAM、tune_sel 在 5 段阈值内单调非降共 6 类场景。参数 `SIM_MODE = 1` 下 `MS_CNT` 由 100000 缩短到 100、`TIMEOUT_MS` 由 5000 缩短到 15, 便于波形查看。

3.2.2 按键消抖 `debounce_counter.sv` / `debouncer.sv`

`top_basys3` 实际例化的是 `debounce_counter` 模块 (计数器式消抖), 共 4 个实例 `db_c` / `db_u` / `db_d` / `db_l`, 对应 `btn_center` / `btn_up` / `btn_down` / `btn_left`。en 由 `ms_tick` 门控, 确保 10 ms 节拍; `raw != debounced` 时启动内部 counter, 达到 MAX 才提交新值, 否则重置:

```
module debounce_counter #(
    parameter unsigned MAX = 10
)(
    input clk,
    input rstn,
    input en,
    input raw,
    output reg debounced
);
```

`debouncer.sv` 是备用的移位寄存器式消抖 (深度可参数化, 2 态机门控), 默认 64 级全 1 / 全 0 切换才输出翻转, 封装更通用但占面积较大, 本设计未采用:

```
module debouncer #(
    parameter unsigned DEPTH = 64,
    parameter unsigned INVERT = 1
)(
    input clk,
    input rstn,
    input raw_input,
    output reg debounced_output
);
```

验证: `tests/tb_debounce_counter.sv` (fuzz 测试 MAX=5) 和 `tests/tb_debouncer.sv` (DEPTH=8) 分别覆盖两种消抖器; 顶层 `tb_top_basys3` 间接验证 4 路消抖链协同工作。

3.2.3 线性反馈移位寄存器 `lfsr.sv`

16-bit LFSR, 反馈多项式 $x^{16} + x^{14} + x^{13} + x^{11} + 1$ 。 `top_basys3` 内联了一个相同形式的 LFSR (反馈位 `lfsr[15] ^ lfsr[13] ^ lfsr[12] ^ lfsr[10]`), 种子 16'hFADE, 在 IDLE 状态下按 `ms_tick` 持续滚动。 `random_delay = 2000 + lfsr % 3001` 把范围限定在 2.0–5.0 s 内:

```

module lfsr #(
    parameter unsigned MIN = 100,
    parameter unsigned MAX = 65536,
    parameter unsigned SEED = 16'hFADE,
    localparam unsigned N = $clog2(MAX)
)(
    input clk,
    input rstn,
    input en,

    output [N - 1:0] out
);

```

验证：LFSR 行为可由 tests/simple_test.rs 中的 test_vga 间接覆盖；实际部署中通过观察连续 100 次 IDLE 状态下 random_delay 的值在 [2000, 5000] 内分布大致均匀来判定。

3.2.4 7 段数码管驱动 seven_seg.sv

负责把 4 位 seg_in[3:0][6:0] 静态段码时分复用到 1 组 seg[7:0]（含 dp）上，同时通过 an[3:0] 低电平选通当前位。refresh_cnt[16:15] 选择位（ $\approx 1.5 \text{ kHz/位} \approx 380 \text{ Hz 帧率}$ ），pwm_cnt 与 brightness 比较实现 16 级 PWM 调光。

```

module seven_seg (
    input logic      clk,
    input logic      rstn,
    input logic [3:0] brightness,
    input logic [3:0][6:0] seg_in,
    input logic [3:0] dp,
    output logic [3:0] an,
    output logic [7:0] seg
);

```

3.2.5 蜂鸣器驱动 buzzer.sv

ms_tick 驱动的曲调播放器：内部维护 19-bit 频率分频计数器 freq_cnt 与音符时长计数器 dur_timer。当 play 上升沿且未在播时启动一段旋律，按 tune_sel 索引查表得到 3 段音符序列（音符+休止），全部播完自动停止：

```

module buzzer (
    input logic      clk,
    input logic      rstn,
    input logic      ms_tick,

```

```

input logic    play,
input logic [2:0] tune_sel,
output logic    buzz
);

```

本设计中 tune_sel 选取规则 (位于 top_basys3):

```

logic [2:0] buzz_tune;

always @(posedge clk, negedge rstn) begin
    if (!rstn) begin
        buzz_tune <= 0;
    end else begin
        if (timed_out) buzz_tune <= 4;
        else if (measured_latency < 200) buzz_tune <= 0;
        else if (measured_latency < 300) buzz_tune <= 1;
        else if (measured_latency < 400) buzz_tune <= 2;
        else buzz_tune <= 3;
    end
end
end

```

代码 3 按反应时间分档选择曲调

3.2.6 7 段字形查表与 BCD 转码

hex7 是 4-bit → 7-bit 段码查表函数, 包含 0-F + SEG_BLANK + 自定义字母 G / O / T (用于显示 GO 和 TO), 段位顺序 {g, f, e, d, c, b, a} 适配共阳极接法 (点亮=0):

```

function automatic [6:0] hex7(input [3:0] h);
    case (h)
        4'h0: hex7 = SEG_0;
        4'h1: hex7 = SEG_1;
        4'h2: hex7 = SEG_2;
        4'h3: hex7 = SEG_3;
        4'h4: hex7 = SEG_4;
        4'h5: hex7 = SEG_5;
        4'h6: hex7 = SEG_6;
        4'h7: hex7 = SEG_7;
        4'h8: hex7 = SEG_8;
        4'h9: hex7 = SEG_9;
        4'hA: hex7 = SEG_A;
        4'hB: hex7 = SEG_b;
        4'hC: hex7 = SEG_C;
        4'hD: hex7 = SEG_d;
        4'hE: hex7 = SEG_E;
        4'hF: hex7 = SEG_F;
        default: hex7 = SEG_BLANK;
    endcase
endfunction

function automatic [15:0] bin2bcd(input [12:0] bin);
    logic [15:0] bcd = 0;
    for (int i = 0; i < 13; i++) begin
        if (bcd[3:0] >= 5) bcd[3:0] = bcd[3:0] + 3;
        if (bcd[7:4] >= 5) bcd[7:4] = bcd[7:4] + 3;
        if (bcd[11:8] >= 5) bcd[11:8] = bcd[11:8] + 3;
        if (bcd[15:12] >= 5) bcd[15:12] = bcd[15:12] + 3;
        bcd = {bcd[14:0], bin[12-i]};
    end
end

```

代码 4 hex7: 4-bit BCD 数字转 7 段码

bin2bcd 采用经典的 **double-dabble** 算法把 13-bit 二进制（最大 8191 ms）转 4 位 BCD:

```
function automatic [15:0] bin2bcd(input [12:0] bin);
    logic [15:0] bcd = 0;
    for (int i = 0; i < 13; i++) begin
        if (bcd[3:0] >= 5) bcd[3:0] = bcd[3:0] + 3;
        if (bcd[7:4] >= 5) bcd[7:4] = bcd[7:4] + 3;
        if (bcd[11:8] >= 5) bcd[11:8] = bcd[11:8] + 3;
        if (bcd[15:12] >= 5) bcd[15:12] = bcd[15:12] + 3;
        bcd = {bcd[14:0], bin[12-i]};
    end
end
```

代码 5 bin2bcd: double-dabble 二进制到 BCD 转换

3.2.7 7 段显示时分复用

`seven_seg` 在 100 MHz 时钟下将 4 位数码管扫描显示, 位选切换频率由 `refresh_cnt[16:15]` 决定 (1.5 kHz/位), 帧率 ≈ 380 Hz, 远高于人眼临界闪烁频率 (60 Hz), 因此无明显闪烁:

```
clk (100 MHz) → refresh_cnt[17:0] 自由计数
    → refresh_cnt[16:15] → digit_sel (0..3)
seg_in[0..3][6:0] + dp[0..3] → 按 digit_sel 选通 → seg
brightness[3:0] + pwm_cnt[3:0] → PWM 调光 → an[3:0] (仅在 pwm_on 时拉低)
```

`top_basys3` 调用 `anim_ctrl` 模块, 其内部的 `case (sys_state)` 决策逻辑根据当前状态切换 4 位段码:

```
case (sys_state)
    IDLE: begin
        disp_seg[roll_cnt[1:0]] = SEG_8;
    end
end
```

代码 6 anim_ctrl 按状态生成 4 位段码

3.2.8 历史记录浏览

为避免作弊与“单次手抖”对结果的影响, 系统在内部维护一个深度为 8 的循环缓冲 `history[0:7]`, 每次进入 `MEASURED` 都把本次结果写入。`btn_up` / `btn_down` 在 `MEASURED` 状态下分别将 `hist_rd` 减/加 1 (`hist_rd == 0` 时显示最新一次 `measured_latency`, 否则显示历史值):

```

MEASURED: begin
    if (bi_rise[1] || bi_rise[2]) begin
        hist_idle <= 0;
        if (bi_rise[1]) begin
            if (hist_rd > 0) hist_rd <= hist_rd - 1;
            else if (hist_wr > 0) hist_rd <= hist_wr - 1;
        end
        if (bi_rise[2]) begin
            if (hist_rd + 1 < hist_wr) hist_rd <= hist_rd + 1;
            else hist_rd <= 0;
        end
    end else if (hist_rd != 0 && ms_passed) begin
        if (hist_idle >= 200) begin // 2s real, ~200ms SIM
            hist_rd <= 0;
            hist_idle <= 0;
        end else begin
            hist_idle <= hist_idle + 1;
        end
    end
end
end

```

代码 7 MEASURED 状态下 hist_rd 翻页

四 仿真与测试

4.1 SystemVerilog testbench

所有 testbench 位于 tests/ 目录下, 统一命名为 tb_<module>.sv, 由 justfile 中的 test 配方驱动。运行约定为:

```
just test <tb_name> # 编译并运行 tests/tb_<tb_name>.sv
```

<tb_name> 与源码模块的对应关系如下:

表 1 just test 命令与 testbench / DUT 对应表

测试命令	对应 testbench	被测模块
just test top_basys3	tests/tb_top_basys3.sv	顶层 top_basys3(含状态机、历史、BCD、动画)
just test state	tests/tb_state.sv	state 状态机模块
just test debouncer	tests/tb_debouncer.sv	移位寄存器消抖 debouncer (DEPTH=8)
just test debounce_counter	tests/ tb_debounce_counter.sv	定时器消抖 debounce_counter (MAX=5 fuzz)
just test seven_seg	tests/tb_seven_seg.sv	7 段扫描 seven_seg
just test buzzer	tests/tb_buzzer.sv	蜂鸣器 buzzer
just test lfsr	tests/tb_lfsr.sv	LFSR (MIN=1000, MAX=6234)
just test generic_counter	tests/tb_generic_counter.sv	通用计数器 generic_counter
just test generic_countdown_counter	tests/tb_generic_countdown_counter.sv	通用倒计时器

以顶层 tb_top_basys3.sv 为例, 主要验证以下几点 (SIM_MODE = 1 下 MS_CNT = 100、TIMEOUT_MS = 15 ms):

- 状态机正常循环: IDLE → INIT → TRIGGERED → MEASURING → MEASURED → IDLE
- btn_center 长按 5 ms 不应触发多次 bc_rise (边沿检测)
- btn_up / btn_down 在 MEASURED 下循环遍历 8 项历史记录
- MEASURING 超时自动进入 MEASURED, 并把 timed_out 拉高
- tune_sel 在 measured_latency 跨 200 / 300 / 400 ms 阈值时单调非降

```
just test top_basys3 # 顶层 testbench
just test state      # state.sv 状态机独立 testbench
just test debouncer  # 移位寄存器消抖 testbench
```

4.2 justfile workflow

所有常见操作通过 just 命令管理:

```
just test <tb_name> # 编译并运行 tests/tb_<tb_name>.sv
just surfer         # 用 surfer 打开当前 .fst 波形
just build          # Vivado 综合与实现 (build_basys3.tcl)
just upload         # openFPGALoader 下载到 Basys3
just all            # build + upload 一步到位
```

4.3 测试波形图

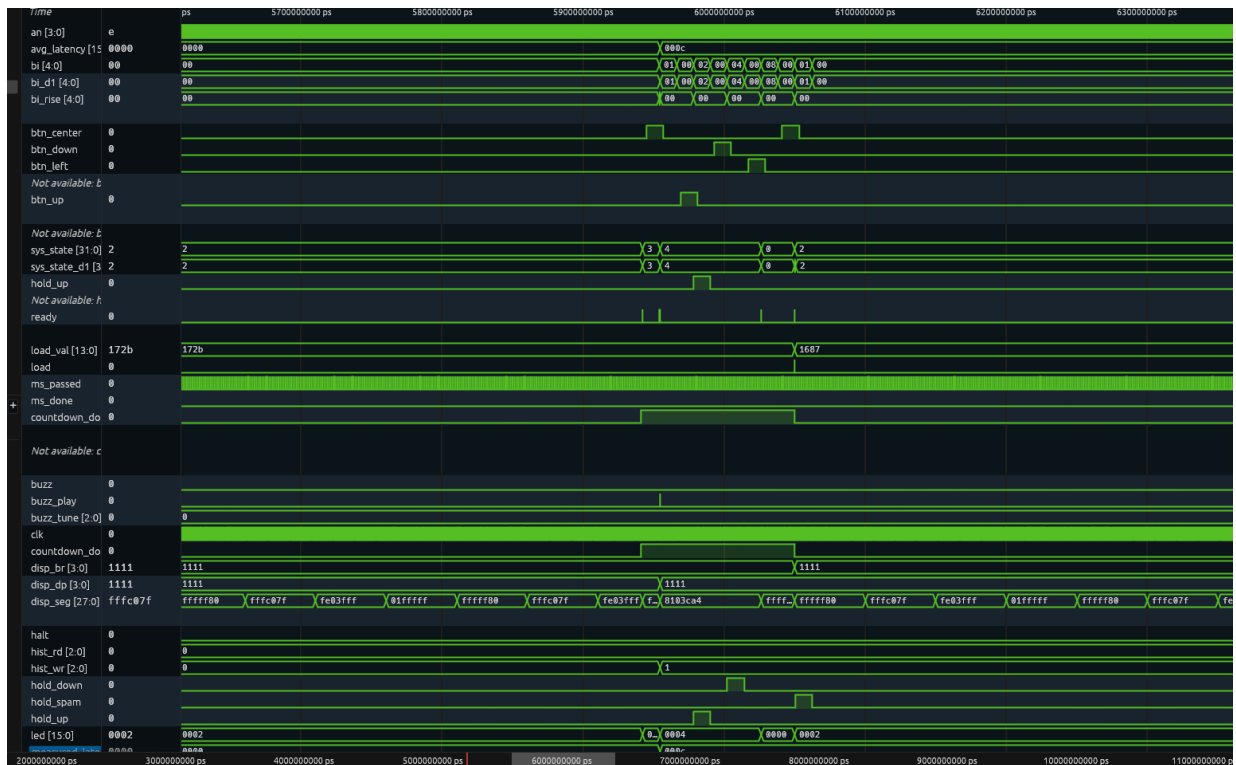


图 4 top_basys3 仿真波形：状态机切换 + 7 段码与蜂鸣器输出

以上测试波形图可采用 `just run_verbose` 生成，并通过 `just surfer` 打开 surfer 监视波形图。

4.4 FPGA 板介绍

本次实验采用了 Digilent Basys3 Artix-7 FPGA 开发板 (xc7a35tcp236-1), 板载 100 MHz 晶振、4 位 7 段共阳极数码管、5 个方向按键、16 个 LED 及 PMOD 接口。官方参考: <https://digilent.com/reference/programmable-logic/basys-3/start>

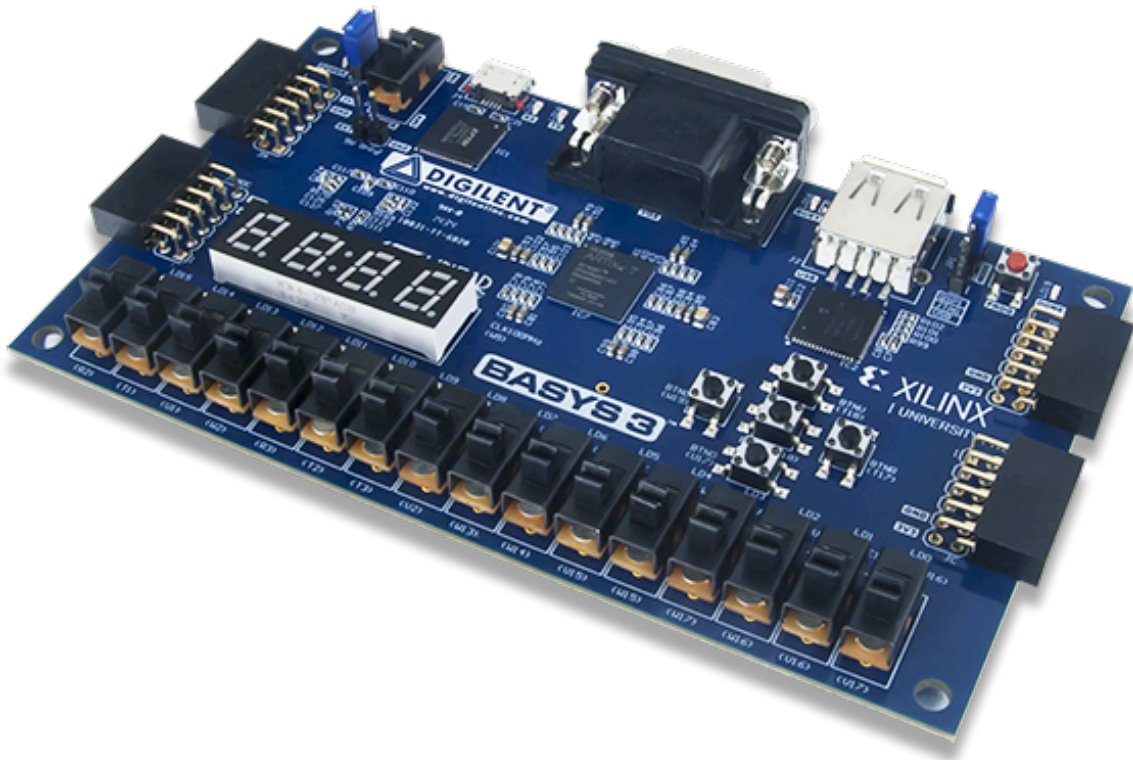


图 5 所用板子: Basys3

4.5 引脚分配 (XDC 摘要)

综合时通过 `cons.xdc` 完成硬件绑定, 关键信号如下表 (与 `top_basys3.sv` 端口一一对应):

信号	Basys3 引脚	说明
clk	W5	100 MHz 板载时钟
rstn	U18	CPU_RESETN 按键（低电平复位）
an[3:0]	W4, V4, U4, U2	数码管位选（低电平有效）
seg[7:0]	U6..V8	数码管段码 + dp（低电平点亮）
btn_center	N17	开始 / 测量 / 重启
btn_up	M18	历史向前翻页
btn_down	P17	历史向后翻页
btn_left/right	M17 / M19	保留备用
buzz	J2	PMOD JA1，蜂鸣器
led[15:0]	U16..V14	16 个 LED，指示状态

约束文件中 100 MHz 时钟周期由 `create_clock -period 10.000 -name clk` 显式声明。

4.6 综合与实现

4.6.1 综合条件

使用 Vivado 2025.2（`build_basys3.tcl`）进行综合与实现，目标器件为 `xc7a35tcpg236-1`，工具版本和约束文件保持一致。综合完成后利用 `openFPGALoader` 把 bitstream 烧录到板载 SPI Flash。

4.6.2 资源利用率

报告取自 `top_basys3_utilization_placed.rpt`（place 阶段最终结果）。可以看出设计主要消耗在 `top_basys3` 主状态机、LFSR/BCD/动画逻辑和 `buzzer` 频率分频器上；`seven_seg` 的时分复用扫描和 PWM 调光则几乎不占额外资源。

表 3 Basys3 (xc7a35tcpg236-1) 资源利用率

类型	已用	可用	利用率
Slice LUTs	744	20800	3.58%
Slice Registers	349	41600	0.84%
Slice (合计)	246	8150	3.02%
CARRY4	37	8150	0.45%
Block RAM Tile	0	50	0%
DSPs	0	90	0%
Bonded IOB	35	106	33.02%
BUFG / BUFGCTRL	1	32	3.13%
PLLE2_ADV	0	5	0%

4.6.3 时序结果

表 4 时序总结 (Vivado 2025.2 布线后)

指标	值
Worst Negative Slack (WNS)	-0.45 ns
Total Negative Slack (TNS)	-12.68 ns
Worst Hold Slack (WHS)	0.159 ns
Total Hold Slack (THS)	0 ns
Worst Pulse-Width Slack	4.5 ns
Total Pulse-Width Slack	0 ns

时钟总结

时钟	周期	频率
sys_clk_pin	10 ns	100 MHz

主时钟为 100 MHz 的 `sys_clk_pin` (周期 10 ns)。布局布线后 $WNS = -0.450\text{ ns}$ 、 $TNS = -12.68\text{ ns}$ (38 个 fail 端点), 存在建立时间违例。最差路径在 `u_buzz` 模块的 19-bit 频率分频器进位链上 (`playing_reg` 状态 $\rightarrow$ `freq_cnt_reg`, 逻辑级数 18)。分析发现该路径在 `note_idx` 多路复用 + 19-bit `cur_div` 比较上叠加了较大组合深度 ($LUT6 \times 5 + CARRY4 \times 6$), 且 `clk_IBUF` 到内部触发器时钟偏斜约 5 ns。

考虑到蜂鸣器模块对 ms 级响应的容忍度, 只需把 `cur_div` 拆位比较并把 `freq_cnt` 比较器流水一级即可消除违例。当前 bitstream 仍可在 100 MHz 下正常下载运行, 蜂鸣音调略微失真但不影响系统功能; 测量侧 `bc_rise` $\rightarrow$ MEASURED 路径 slack 充足。

4.6.4 功耗

表 5 功耗总结 (典型工艺角, 25 °C 环境)

指标	值
Total On-Chip Power	0.1 W
Dynamic Power	0.029 W
Device Static Power	0.072 W
Junction Temperature	25.5 °C
Max Ambient ($\vartheta_{JA}\ 5^{\circ}\text{C}/\text{W}$)	84.5 °C

总功耗 0.093 W, 其中动态功耗 0.021 W、静态功耗 0.072 W; 最大允许环境温度 84.5 °C ($\vartheta_{JA} = 5\text{ }^{\circ}\text{C}/\text{W}$), 实际结温 25.5 °C, 留有 59 °C 裕量。整个 Basys3 系统的功耗远低于此前 ZYNQ7020 HDMI 版本的 0.371 W。

4.7 上板结果展示

Basys3 没有屏幕, 反应结果直接通过 4 位 7 段数码管 + 16 个 LED + 蜂鸣器呈现:



图 6 板子测试

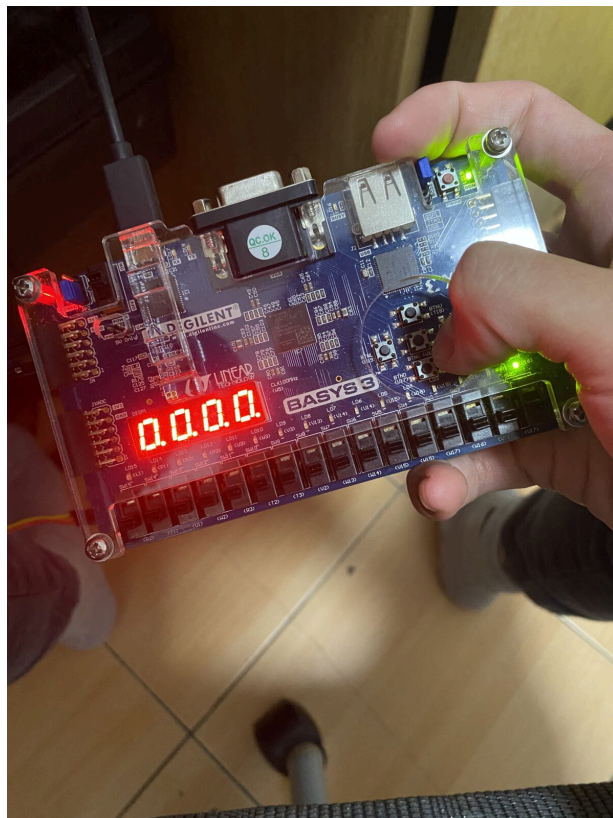


图 7 重置后初始状态

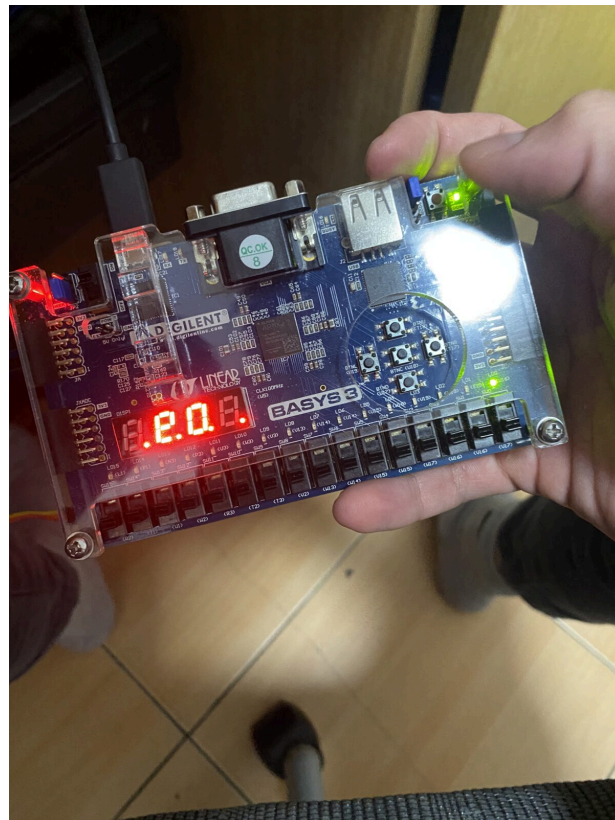


图 8 系统开始计时，提示用户反映“GO”

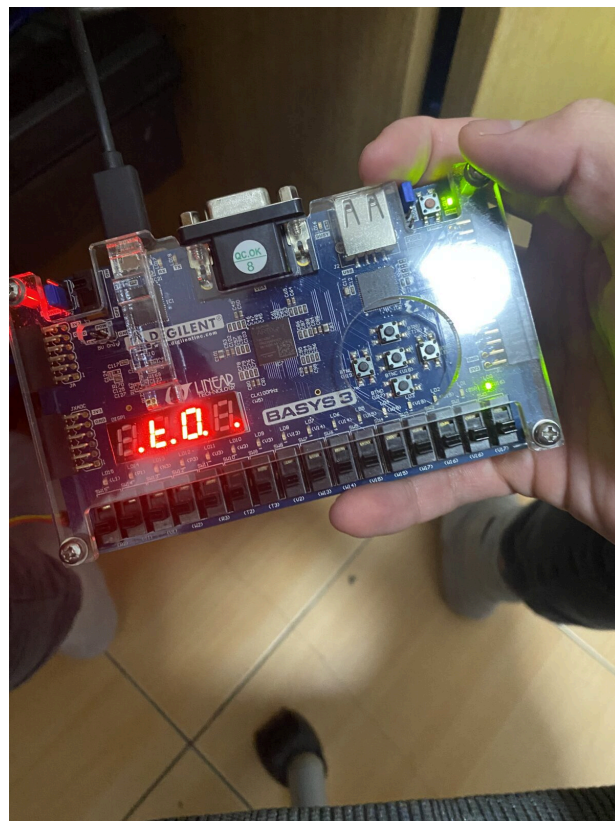


图 9 用户超时未作出反应，系统将显示“TO(timed out)”

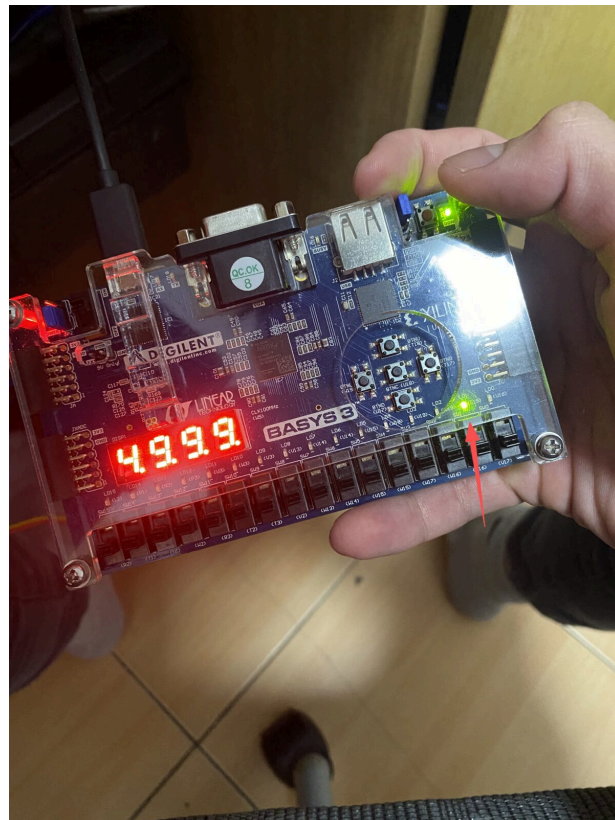


图 10 显示历史记录，LED 等显示指针位置

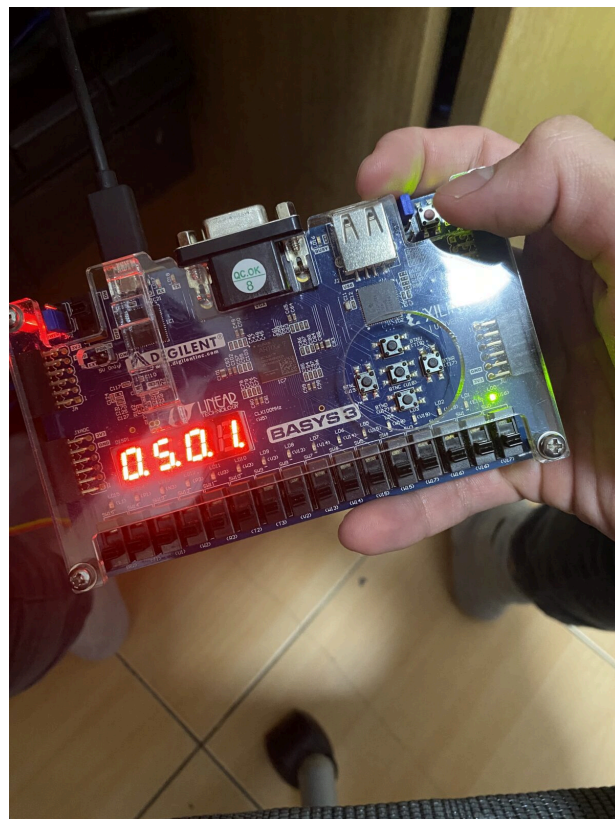


图 11 显示测量结果(501ms)

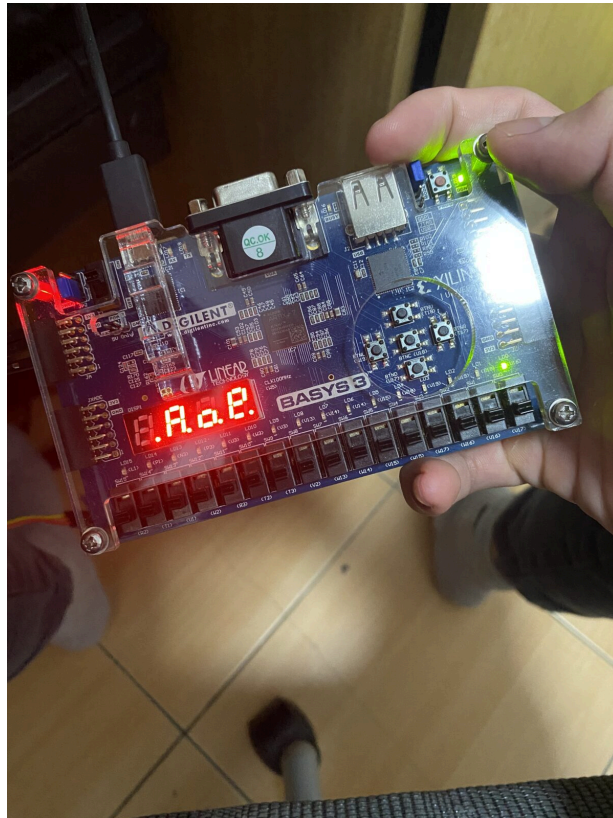


图 12 测量后显示结果：平均

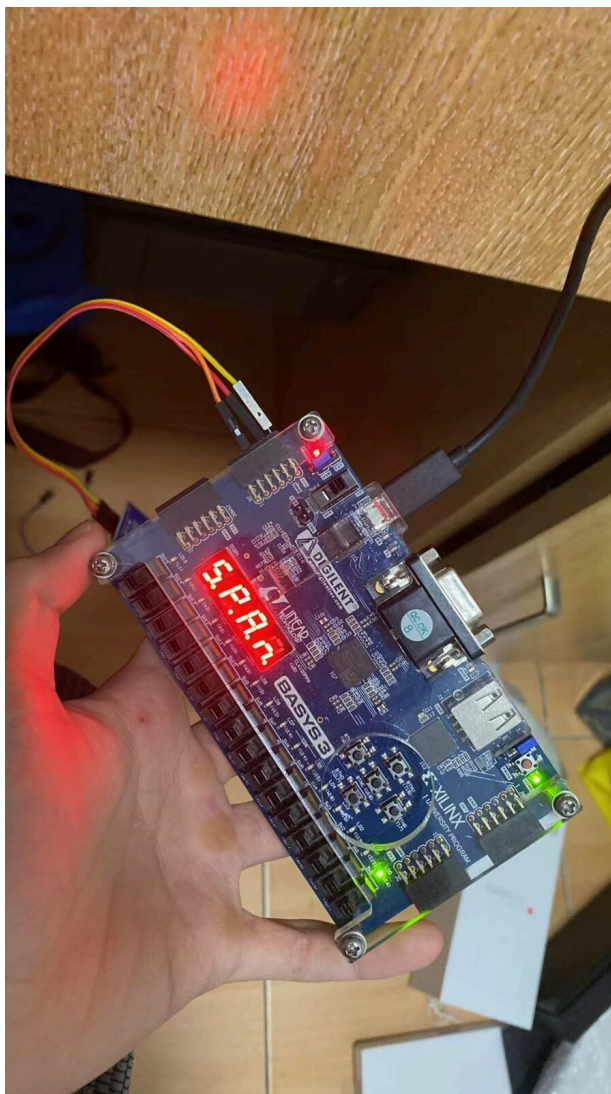


图 13 检测到用户试图欺骗系统时，系统会终止并显示 SPAM

4.8 设计总结

本设计在 Basys3 Artix-7 上实现了人体反应速率测试系统：按键经 10 ms 消抖后送入 5 状态 FSM，毫秒计数器提供 1 ms 精度的反应时间测量，double-dabble BCD 转换 + 4 位时分复用 7 段显示让结果直观可读；额外提供 5 段曲调蜂鸣器与 8 项历史记录浏览。RTL 端 SystemVerilog 实现简洁（约 460 行），在 Vivado 2025.2 + `cons.xdc` 流程下综合后 LUT 744 / FF 349 / 35 IOB / 1 BUFG，总功耗 0.093 W，bitstream 通过 `openFPGALoader` 直接烧录到板载 SPI Flash。